

Veille Prompt Engineering

Document de synthèse - P1IA - Groupe 1

Septembre 2026

Sommaire	
1. Prérequis sur les LLM	
2. Introduction au Prompt Engineering	
3. Structure d'un prompt	
4. Techniques de prompting	
5. Prompt Engineering et raisonnement	
6. Sorties structurées	
7. Prompts pour les applications métier	
8. Prompts pour le Machine Learning	
9. Prompt Engineering et RAG	
10. Évaluation et optimisation des prompts	
11. Limites et risques	
12. Références	

1. Prérequis sur les LLM

Un **LLM** (Large Language Model, modèle de langage de grande taille) est un programme d'intelligence artificielle entraîné sur d'énormes quantités de texte (livres, sites web, documents...) pour **prédire le mot suivant** dans une phrase. Sa connaissance vient de son entraînement : il ne « sait » pas, il calcule les mots les plus probables à chaque étape.

1.1 Fonctionnement général

- **Contexte** : tout le texte déjà présent dans la conversation ou dans le prompt. Le modèle s'appuie dessus pour générer la suite.
- **Token** : unité de base. Un token n'est pas un mot complet : il peut être un mot, une partie de mot ou un signe de ponctuation. En moyenne, 100 tokens $\approx$ 75 mots.
- **Fenêtre de contexte** : nombre maximal de tokens que le modèle peut « voir » (entrée + sortie). Ex. 8 000 tokens $\approx$ 6 000 mots. Au-delà, le début du texte est oublié / rejeté.
- **Génération** : le modèle produit un token à la fois et s'arrête à un token spécial de fin, ou quand le nombre maximal autorisé est atteint.

1.2 Paramètres qui influencent les réponses

Paramètre	Rôle	Valeur conseillée
Temperature	Contrôle le hasard de la réponse. Basse = réponse fiable et répétitive. Haute = réponse créative, variée.	0,1 - 0,3 (fiable) 0,7 - 1,0 (créatif)
max tokens	Limite la longueur de la sortie.	Assez grand pour la tâche
Top-p	Coupe de l'échantillonnage : ne garde que les tokens dont la probabilité cumulée atteint p. Faible = déterministe.	0,1 - 1,0
Top-k	Ne considère que les k tokens les plus probables à chaque pas.	10 - 50
stop sequences	Séquences qui stoppent la génération (ex. fin de liste, balise JSON).	Selon le format

Règle simple : **si la tâche demande précision** → **temperature basse (ex. 0,2)** ; **si elle demande imagination** → **temperature plus haute (ex. 0,8)**.

2. Introduction au Prompt Engineering

2.1 Qu'est-ce qu'un prompt ?

Un **prompt** est l'instruction que l'on donne à un modèle d'IA : une question, un ordre, un texte à transformer. C'est l'unique façon de « piloter » le modèle.

2.2 Qu'est-ce que le Prompt Engineering ?

Le **Prompt Engineering** est la discipline qui consiste à **concevoir, tester et améliorer des prompts** pour obtenir des réponses précises, fiables et exploitables. Ce n'est pas de la programmation : c'est un savoir-faire de rédaction et de méthode.

2.3 Pourquoi est-ce important ?

- Un bon prompt peut **doubler la qualité** d'une réponse sans changer de modèle.
- Cela **économise du temps** : moins d'allers-retours, moins de corrections.
- Cela **réduit les erreurs** : hallucinations, oublis, formats inexploitable.
- Cela permet d'industrialiser l'IA dans les applications métier.

Mauvais prompt : Résume ce texte. Bon prompt : Tu es un assistant qui rédige des synthèses pour des cadres pressés. Résume le texte ci-dessous en 5 puces, maximum 30

mots chacune. Commence chaque puce par le mot-clé principal en gras.

3. Structure d'un prompt

Un prompt efficace se construit avec des **briques réutilisables**. Les délimiteurs (guillemets, triples chevrons, balises XML) permettent au modèle de distinguer les instructions du texte à traiter.

Brique	Rôle	Exemple
Rôle	Donne une posture au modèle : expert, relecteur, formateur...	« Tu es un data analyst sénior. »
Contexte	Situation, destinataire, contraintes de fond.	« Public : débutants en Excel. »
Tâche	L'action demandée (verbe d'action).	« Explique la fonction RECHERCHEV. »
Données d'entrée	Le texte / les données à traiter, clairement délimitées.	« Texte : <texte> »
Contraintes	Règles : longueur, interdits, style, langue.	« 10 lignes max, sans jargon. »
Format de sortie	La structure attendue du résultat.	« Tableau Markdown à 3 colonnes. »

Exemple complet :

Tu es un analyste financier. # ROLE Synthèse pour un PDG non technique. # CONTEXTE Résume ce rapport trimestriel. # TACHE Rapport : <<<texte>>> # DONNEES Maximum 8 lignes, sans jargon. # CONTRAINTES Format : 3 puces + 1 conclusion. # FORMAT

4. Techniques de prompting

4.1 Zero-shot prompting

Aucun exemple. On décrit la tâche et on fournit l'entrée. Idéal pour les tâches simples que le modèle connaît bien.

Classe le sentiment de ce commentaire (positif / négatif / neutre) : « Le produit est arrivé cassé. »

4.2 One-shot prompting

Un seul exemple montre au modèle le format ou le style attendu.

Convertis un titre de film en emojis. Exemple : « Jaws » -> [requin], [vague] « Titanic » -> ?

4.3 Few-shot prompting

Plusieurs exemples (généralement 2 à 5). Le modèle détecte le « schéma » et l'applique au nouveau cas. Puissant pour les formats inhabités et les classifications.

Classe chaque e-mail : Travail / Personnel / Spam. « Promotion -50% ce week-end ! » -> Spam « Peux-tu m'envoyer le budget Q2 ? » -> Travail « Recette de tarte de grand-mère » -> ?

4.4 Chain-of-Thought (CoT)

On demande au modèle de **raisonner étape par étape** avant de conclure. Cela améliore fortement les résultats sur les problèmes de logique, de calcul et de raisonnement.

Un train part de Paris à 9h à 120 km/h. Un second part de Lyon à 9h à 90 km/h. Paris-Lyon : 470 km. À quelle heure se croisent-ils ? Raisonne étape par étape.

4.5 Structured prompting (prompting structuré)

La sortie est contrainte à un **format précis** (JSON, XML, tableau, sections). Facilite l'exploitation automatique des réponses.

Réponds exclusivement en JSON, sans texte autour : { "resume": "...", "mots_cles": ["...", "..."], "score_confiance": 0.9 }

5. Prompt Engineering et raisonnement

5.1 Prompt decomposition (décomposition)

Casser un problème complexe en **sous-problèmes traités séparément**. Chaque sous-tâche est simple, donc plus fiable. On assemble ensuite les résultats.

Étape 1 : extrais la liste des produits et leur prix. Étape 2 : pour chaque produit, calcule la marge. Étape 3 : classe les produits par marge décroissante.

5.2 Self-check / vérification

Le modèle **vérifie sa propre réponse**, puis la corrige. On peut aussi générer la réponse, puis demander une critique indépendante, puis fusionner.

1. Propose une réponse. 2. Relis ta réponse et liste 3 erreurs ou oublis. 3. Corrige la réponse en tenant compte de ces remarques.

6. Sorties structurées

6.1 Pourquoi structurer ?

- Facilite l'exploitation automatique (bases de données, tableaux de bord).
- Uniformise les réponses → plus faciles à comparer et à valider.
- Réduit les oublis : la structure impose les champs.

6.2 Formats

Format	Usage
JSON	Échange de données avec les applications (API, automatisation).
XML	Documents éditoriaux, catalogues, échanges normalisés.
Tables (Markdown/CSV)	Tableaux à plat, exports vers Excel.
Sections / titres	Rapports et documents lisibles.

6.3 Validation des sorties

- Vérifier « hors du prompt » : le code, pas le modèle, vérifie que le JSON est valide et que les champs obligatoires existent.
- Demander au modèle de se relire face à un schéma de référence.
- Retenter avec un prompt corrigé si la validation échoue.

7. Prompts pour les applications métier

Application	Objectif	Prompt type
Résumé de documents	Compresser l'essentiel d'un texte.	« Résume ce document en 3 puces, en gardant les chiffres clés. »
Traduction	Convertir sans perdre le sens ni le ton.	« Traduis en anglais professionnel, garde la ponctuation et les chiffres. »
Classification	Ranger dans des catégories fixées.	« Classe ce ticket : bug / question / demande de fonctionnalité. »

Extraction d'informations	Récupérer des données précises. « Extraits : nom, date, montant. Réponds en JSON. »
Génération de texte	Créer des contenus (mails, articles). « Rédige un mail de relance, 150 mots, ton courtois. »
Génération de code	Produire ou corriger du code. « Écris une fonction Python qui calcule la moyenne d'une liste. »

Règle d'or : toujours **préciser le public, la longueur, le ton et le format de sortie**. « Un mail » ne suffit pas : « un mail de relance de paiement au fournisseur X, 150 mots, ton courtois, avec date limite et références de la facture » donne une réponse utilisable directement.

8. Prompts pour le Machine Learning

8.1 Préparation des données

Nettoie ce jeu de données : supprime les doublons, corrige les valeurs abérantes, normalise les dates et réponds par un tableau indiquant chaque correction apportée.

8.2 Génération de visualisations

Génère le code Python (matplotlib) d'un histogramme du CA mensuel à partir de ces données. Affiche les étiquettes en français, taille 800x500.

8.3 Sélection de modèles

Je dois classer des e-mails (spam / non-spam), 10 000 exemples. Compare 3 familles de modèles et recommande-en une, en je précisant le compromis précision / coût / simplicité.

8.4 Interprétation des métriques

Voici une matrice de confusion : précision 0,72, rappel 0,88. Explique-moi ce que cela signifie pour un dépistage médical (FP, FN) et propose 2 pistes d'amélioration.

9. Prompt Engineering et RAG

9.1 Pourquoi le prompting seul ne suffit pas toujours ?

- Le modèle ne connaît que son entraînement : il ignore vos documents internes et l'actualité.
- Il peut mélanger plusieurs sujets quand le contexte manque.
- Sans source, impossible de vérifier la réponse.

9.2 Prompt + contexte documentaire — le RAG

Le **RAG** (Retrieval-Augmented Generation, génération augmentée par récupération) consiste à : **1)** chercher dans une base de documents les morceaux pertinents, **2)** les insérer dans le prompt, **3)** demander au modèle de répondre **uniquement à partir de ces extraits**.

9.3 Construction d'un prompt RAG

Réponds **UNIQUEMENT** à partir des extraits ci-dessous. Si l'information n'y figure pas, réponds : « Information introuvable dans les documents. » Extraits : <<<doc1 : ...>>><<<doc2 : ...>>> Question : ... Réponds en français, 200 mots max, avec les sources citées.

10. Évaluation et optimisation des prompts

10.1 Pourquoi évaluer ?

Un prompt « qui marche une fois » peut échouer sur d'autres entrées. **L'évaluation** mesure la qualité de façon reproductible avant de déployer.

10.2 Mesure de la qualité des réponses

- Critères : exactitude, exhaustivité, format, ton, absence de hallucination.
- Méthodes : grille de notation manuelle, tests sur un échantillon fixe, comparaison par un autre modèle, métriques (F1, Cohen's kappa...).

10.3 Comparaison de résultats

Tester deux prompts sur les mêmes entrées et comparer côte à côte les sorties (A/B testing). Notez-les avec une grille identique.

10.4 Optimisation itérative

Cycle : 1) écrire le prompt 2) témoigner sur Méfiance échantillon 3) identifier les erreurs 4) corriger le prompt 5) re-tester. Recommencer jusqu'à stabilité.

Garde une trace des versions (version 1, version 2...) : c'est ce qui permet de progresser en connaissance de cause.

11. Limites et risques

Risque	Définition	Mitigation
Hallucinations	Réponses fausses mais présentées avec confiance	Assurer la provenance des citations + validation humaine.
Ambiguïté	Prompt flou → réponse aléatoire ou hors sujet	Préciser tâche, format, périmètre ; reformuler.
Biais	Reproduit des stéréotypes présents dans les données	Respecter les chartes, évaluation sur des cas délicats, relecture.
Confidentialité	Les données envoyées peuvent être stockées par le fournisseur	Né jamais envoyer de données sensibles ; anonymiser.
Injection de prompt	Attaque : des données utilisateur désactivent les instructions	Ne pas insérer d'instructions et données ; validateurs.
Jailbreak	Contournement des garde-fous de sécurité	Filter le modèle, supervision, mises à jour de sécurité.

Le prompt engineering ne supprime pas les risques : il les **réduit**. La règle de base reste la **supervision humaine** sur les décisions importantes.

12. Références

- OpenAI - Prompt Engineering Guide : <https://platform.openai.com/docs/guides/prompt-engineering>
- Anthropic - Prompt Engineering Overview : <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering>
- Google Cloud / Google Developers - Prompting guide (cheat sheet) : <https://ai.google.dev/gemini-api/docs/prompting-intro>
- Prompt Engineering Guide (DAIR AI) : <https://www.promptingguide.ai>
- Microsoft - Azure OpenAI best practices : <https://learn.microsoft.com/fr-fr/azure/ai-services/openai/concepts/prompt-engineering>
- Report on CoT - « Chain-of-Thought Prompting Elicits Reasoning in LLMs » (Wei et al., 2022) : <https://arxiv.org/abs/2201.11903>
- RAG - « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks » (Lewis et al., 2020) : <https://arxiv.org/abs/2005.11401>

- Learn Prompting : <https://learnprompting.org>
- Cheat sheets collectés (dossier du livrable) : <https://ipwithease.com>, <https://www.rpatech.ai>,
Google Prompting guide